

Guía de Experto sobre el Java Collections Framework: Optimización y Casos de Uso

1.0 Fundamentos del Framework: Los Contratos de Colección

El Java Collections Framework (JCF) es un conjunto robusto de clases e interfaces que proporcionan estructuras de datos listas para usar, diseñadas para almacenar y manipular grupos de objetos de manera eficiente.¹ La arquitectura fundamental del JCF es su principal ventaja: la separación del contrato (la interfaz) de su implementación (la clase).¹

1.1 La Arquitectura de Interfaces (El "Qué" vs. el "Cómo")

El JCF se basa en un conjunto de interfaces que definen el "qué": los comportamientos abstractos que una colección debe proporcionar (por ejemplo, "agregar un elemento" o "encontrar un elemento"). Las clases concretas, como `ArrayList` o `HashMap`, proporcionan el "cómo": la implementación específica de la estructura de datos (por ejemplo, un array dinámico o una tabla hash).¹

Esta separación es la piedra angular para escribir código desacoplado y mantenible. Permite a un desarrollador cambiar la implementación subyacente (por ejemplo, reemplazar un `ArrayList` por un `LinkedList`) para optimizar el rendimiento en respuesta a nuevos perfiles de uso, sin necesidad de alterar el código de la lógica de negocio que depende del contrato de la interfaz. Este principio se explorará en detalle en la Sección 8.1.

1.2 Interfaces Raíz: `Collection` vs. `Map`

El framework tiene dos jerarquías de interfaces distintas:

- **`java.util.Collection<E>`:** Esta es la interfaz raíz para cualquier "grupo de objetos".³ Define operaciones universales como `add(E e)`, `remove(Object o)`, `contains(Object o)`, `size()` e `iterator()`.²

- **java.util.Map<K, V>**: Esta interfaz define un "mapeo de claves a valores".³ Es conceptualmente distinta y no extiende Collection. Un Map no es una colección de elementos; es una colección de *asociaciones* (pares clave-valor).⁵ Mientras Collection responde a la pregunta "¿Contiene este objeto?", Map responde a "¿Qué valor está asociado con esta clave?". Los Map proporcionan "vistas de colección" (keySet(), values(), entrySet()) como un puente para permitir que sus componentes sean tratados como colecciones.⁶

1.3 Sub-interfaces Principales de Collection

La interfaz Collection se especializa en tres contratos principales:

- **java.util.List<E>**: Representa una colección *ordenada*, también conocida como secuencia.² Este contrato garantiza que los elementos se mantienen en un orden específico (generalmente el orden de inserción) y permite el acceso posicional a través de un índice numérico.⁴ Las implementaciones de List permiten elementos duplicados.²
- **java.util.Set<E>**: Representa una colección que *no contiene elementos duplicados*.² Este contrato modela la abstracción del conjunto matemático, imponiendo la unicidad de sus elementos.⁹
- **java.util.Queue<E>**: Define una colección diseñada para *mantener elementos antes de procesarlos*.² Aunque no es una garantía estricta, la mayoría de las colas siguen un orden FIFO (First-In, First-Out).¹
- **java.util.Deque<E>**: Una "cola de doble extremo" (*double ended queue*) que extiende Queue.³ Su contrato permite la inserción y eliminación eficiente de elementos en *ambos* extremos: la cabeza y la cola.²

2.0 Análisis Profundo de List: Orden y Acceso Posicional

La elección de una implementación de List es una de las decisiones más comunes y con mayor impacto en el rendimiento. Las dos implementaciones principales, ArrayList y LinkedList, tienen perfiles de rendimiento diametralmente opuestos.

2.1 java.util.ArrayList: El Poder del Acceso Aleatorio

ArrayList es la implementación de List respaldada por un array dinámico (redimensionable).⁸ Implementa la interfaz marcadora RandomAccess, una señal clara de que sus operaciones de acceso posicional son extremadamente rápidas.¹⁴

Análisis de Rendimiento (Notación Big O):

- **get(int index) / set(int index, E e): $O(1)$ (Tiempo constante).**¹⁴ Este es su punto fuerte. Debido a que los datos se almacenan en un array contiguo, la JVM puede calcular la dirección de memoria de cualquier elemento instantáneamente usando aritmética de punteros ((dirección-base + índice * tamaño-elemento)).⁸
- **add(E e) (al final): $O(1)$ Amortizado.**¹⁴ La adición de un elemento es $O(1)$ siempre que haya espacio en el array subyacente. Cuando se excede la capacidad, se debe crear un nuevo array (más grande) y todos los elementos se copian, una operación $O(n)$.⁸ Sin embargo, como esto ocurre con poca frecuencia, el costo promedio (amortizado) a lo largo de n inserciones se mantiene en $O(1)$.¹⁴
- **add(int index, E e) / remove(int index): $O(n)$ (Tiempo lineal).**¹⁵ Para insertar o eliminar en el medio, todos los elementos a la derecha del índice deben ser desplazados (copiados) una posición.¹³ Aunque esta operación es muy rápida a nivel de hardware (utilizando System.arraycopy() ⁸), sigue siendo algorítmicamente $O(n)$.

2.2 java.util.LinkedList: El Especialista en Manipulación de Extremos

LinkedList implementa List (y Deque) utilizando una lista doblemente enlazada. Cada elemento es un "nodo" que almacena punteros a sus vecinos "anterior" y "siguiente".⁸

Análisis de Rendimiento (Notación Big O):

- **get(int index) / set(int index, E e): $O(n)$ (Tiempo lineal).**¹⁵ Este es su punto débil. Para encontrar el elemento en el índice 2,500,000, LinkedList debe recorrer 2,500,000 nodos desde el principio (o el final, el que esté más cerca).¹⁵ No hay acceso aleatorio.¹⁸
- **addFirst() / addLast() / removeFirst() / removeLast(): $O(1)$ (Tiempo constante).**¹⁶ La manipulación en los extremos es instantánea; solo implica actualizar los punteros del primer o último nodo.
- **add(int index, E e) / remove(int index): $O(n)$ (Tiempo lineal).**¹⁶ Es un error común pensar que LinkedList es $O(1)$ para la inserción en el medio. La operación de *vinculación* (cambiar punteros) es $O(1)$ ¹⁵, pero *primero* se debe *encontrar* el nodo en index, lo cual es una operación de recorrido $O(n)$.¹⁵
- **Iterator.remove() / ListIterator.add(E e): $O(1)$ (Tiempo constante).**¹⁶ Este es el único caso de uso de nicho donde LinkedList brilla para la manipulación. Si ya se está iterando sobre la lista y se tiene una referencia al nodo actual (a través del Iterator), la inserción o eliminación *en esa posición exacta* es $O(1)$ porque no se requiere ningún recorrido.¹⁶

2.3 Veredicto: ArrayList vs. LinkedList

Use ArrayList. En casi todos los escenarios del mundo real, ArrayList es la opción preferida.¹⁶

Aunque ambas implementaciones tienen una inserción/eliminación $O(n)$ en el medio, la $O(n)$ de ArrayList (un `System.arraycopy()`) es frecuentemente más rápida que la $O(n)$ de LinkedList (un recorrido de punteros).⁸ Esto se debe a dos factores: `System.arraycopy()` es una operación de hardware altamente optimizada, y ArrayList tiene una "localidad de caché" (cache locality) superior. Los elementos de un array están contiguos en la memoria, lo que permite que la CPU los cargue en el caché de manera muy eficiente. Los nodos de LinkedList están dispersos por el heap, lo que provoca constantes "fallos de caché" (cache misses).¹⁹ LinkedList solo debe considerarse en dos casos:

1. Como implementación de Deque (Pila o Cola), donde todas las operaciones ocurren en los extremos ($O(1)$).¹⁶ (Aunque, como se verá en la Sección 5.0, `ArrayDeque` es incluso mejor para esto).
2. En aplicaciones muy específicas que realizan una gran cantidad de inserciones/eliminaciones en medio de la lista *utilizando exclusivamente iteradores*.¹⁶

La siguiente tabla resume las compensaciones de rendimiento que dictan la elección entre ArrayList y LinkedList.

Tabla 2.1: Comparativa de Complejidad Algorítmica de List

Operación	ArrayList	LinkedList	Justificación de Rendimiento
get(index)	$O(1)$	$O(n)$	ArrayList usa acceso aleatorio (indexación). ¹⁴ LinkedList debe recorrer desde un extremo. ¹⁶
add(E) (al final)	$O(1)$ Amortizado	$O(1)$	ArrayList puede requerir un redimensionamiento $O(n)$. ¹⁴ LinkedList solo actualiza el puntero final. ¹⁶
add(index, E) (medio)	$O(n)$	$O(n)$	ArrayList requiere desplazamiento de array (<code>System.arraycopy()</code>). ¹⁵ LinkedList requiere recorrido $O(n)$ para encontrar el índice. ¹⁵
remove(index) (medio)	$O(n)$	$O(n)$	ArrayList requiere desplazamiento de array. ¹⁵ LinkedList

			requiere recorrido $O(n)$ para encontrar el índice. ¹⁶
<code>Iterator.remove()</code>	$O(n)$	$O(1)$	ArrayList aún debe desplazar los elementos restantes. ¹⁶ LinkedList (con el iterador en posición) solo re-vincula punteros. ¹⁶

3.0 Análisis Profundo de Set: Garantía de Unicidad

La interfaz Set se utiliza cuando el requisito principal es la unicidad.⁹ La elección de la implementación de Set no se basa en el acceso posicional (ya que no existe), sino en las garantías de *orden* y el *rendimiento* de las comprobaciones de membresía.

3.1 java.util.HashSet: El Estándar $O(1)$

- **Implementación Interna:** Utiliza una instancia de HashMap.⁸ Los elementos del Set se almacenan como *claves* en el HashMap subyacente, y un objeto ficticio se usa como valor.
- **Orden: Ninguno garantizado.**¹⁰ El orden de iteración es impredecible.
- **Rendimiento: $O(1)$ (Tiempo constante)** para `add()`, `remove()` y `contains()`.¹⁷
- **Manejo de Nulos:** Permite un único elemento null.¹⁰
- **Caso de Uso:** La implementación de Set de propósito general y más rápida. Debe usarse siempre que se necesite unicidad y no se requiera un orden específico.²¹

3.2 java.util.LinkedHashSet: Unicidad con Orden de Inserción

- **Implementación Interna:** Extiende HashSet. Utiliza un LinkedHashMap subyacente, que combina una tabla hash con una lista doblemente enlazada.¹⁰
- **Orden: Orden de inserción.**¹⁰ Los elementos se iteran en el orden en que se agregaron.
- **Rendimiento: $O(1)$ (Tiempo constante)** para `add()`, `remove()` y `contains()`.¹⁰ Es marginalmente más lento que HashSet debido a la sobrecarga de mantener la lista enlazada.²¹

- **Manejo de Nulos:** Permite un único elemento null.¹⁰
- **Caso de Uso:** Cuando se necesita la unicidad de un Set pero también se debe preservar el orden de entrada (por ejemplo, una lista de reproducción sin duplicados).²¹

3.3 java.util.TreeSet: Unicidad con Orden Natural

- **Implementación Interna:** Utiliza una instancia de TreeMap²⁴, que se implementa como un árbol Rojo-Negro (un árbol de búsqueda binario auto-balanceado).¹⁰
- **Orden: Ordenado.**¹⁰ Los elementos se ordenan según su *orden natural* (si la clase implementa Comparable) o mediante un Comparador proporcionado en el constructor.⁸
- **Rendimiento: $O(\log n)$** para add(), remove() y contains().¹⁰
- **Manejo de Nulos: No permite null.**¹⁰ Se lanzará una NullPointerException al intentar agregar null, ya que null no se puede comparar.
- **Caso de Uso:** Cuando se necesita un conjunto de elementos únicos que *siempre* esté ordenado (por ejemplo, un directorio de usuarios ordenado alfabéticamente).²³

3.4 Caso de Estudio: Set.contains() vs. List.contains() para Eficiencia

Una operación de aplicación común es verificar si una colección ya contiene un elemento (contains(Object o)). La elección de la estructura de datos aquí tiene implicaciones drásticas en el rendimiento.

- **ArrayList.contains(): Rendimiento $O(n)$.**¹⁷ Para determinar si la lista contiene el objeto, debe iterar linealmente sobre cada elemento y ejecutar el método equals() en cada uno.²⁰
- **HashSet.contains(): Rendimiento $O(1)$.**¹⁷ No itera. Utiliza el hashCode() del objeto para saltar instantáneamente al *bucket* (cubeta) correcto donde *debería* estar el elemento, y luego usa equals() solo en los pocos elementos dentro de ese bucket.²⁰

Si una aplicación realiza con frecuencia operaciones contains() en una List grande, especialmente dentro de un bucle, el rendimiento algorítmico será $O(n^2)$ (o peor), lo que puede paralizar la aplicación.

Mejor Práctica: Si se requiere un gran número de comprobaciones de membresía, se debe **convertir la List a un HashSet primero**. Esta conversión tiene un costo único de $O(n)$. Después de eso, cada una de las miles (o millones) de comprobaciones contains() subsiguientes se reduce de $O(n)$ a $O(1)$. Los benchmarks confirman que HashSet es órdenes de magnitud más rápido para esta operación.²⁰

Tabla 3.1: Comparativa de Implementaciones de Set

Implementación	Implementación Interna	Rendimiento (Big O)	Orden de Iteración	Permite Nulos
----------------	------------------------	---------------------	--------------------	---------------

HashSet	HashMap	$O(1)$	Ninguno garantizado ¹⁰	Sí (uno) ¹⁰
LinkedHashSet	LinkedHashMap	$O(1)$	Orden de inserción ¹⁰	Sí (uno) ¹⁰
TreeSet	TreeMap (Árbol Rojo-Negro)	$O(\log n)$	Ordenado (Natural o Comparator) ¹⁰	No ¹⁰

4.0 Análisis Profundo de Map: Mapeo de Clave-Valor

Las implementaciones de Map son la base de las implementaciones de Set y, por lo tanto, sus perfiles de rendimiento y orden son paralelos. La elección se centra en cómo se deben gestionar y acceder las claves.

4.1 java.util.HashMap: El Estándar $O(1)$

- **Implementación Interna:** Una tabla hash (array de buckets).²⁶
- **Orden:** Ninguno garantizado.²⁶ El orden de iteración de claves/valores es impredecible.
- **Rendimiento:** **$O(1)$ (Tiempo constante)** para put(), get() y containsKey().²⁶
- **Manejo de Nulos:** Permite una clave null y múltiples valores null.²⁶
- **Caso de Uso:** La implementación de Map más rápida y de uso más general. Se debe usar siempre que se necesite una asociación clave-valor y el orden no importe.²⁹

4.2 java.util.LinkedHashMap: Orden de Inserción

- **Implementación Interna:** Una tabla hash combinada con una lista doblemente enlazada que conecta todas las entradas.²⁶
- **Orden:** Orden de inserción (o, opcionalmente, "orden de acceso").²⁶
- **Rendimiento:** **$O(1)$ (Tiempo constante)** para put(), get() y containsKey().²⁶
- **Manejo de Nulos:** Permite una clave null y múltiples valores null.²⁶
- **Caso de Uso:** Esencial cuando se necesita la velocidad de HashMap pero se requiere un orden de iteración predecible.³⁰ También es la base perfecta para construir cachés LRU (Least Recently Used).³⁰

4.3 java.util.TreeMap: Orden de Claves

- **Implementación Interna:** Un árbol Rojo-Negro.²⁶
- **Orden: Ordenado** por la clave (según el orden natural o un Comparator).²⁶
- **Rendimiento:** $O(\log n)$ para put(), get() y containsKey().²⁶
- **Manejo de Nulos: No permite claves null** (lanzará NullPointerException).²⁷ Permite valores null.²⁷
- **Caso de Uso:** Cuando se necesita un mapa que siempre esté ordenado por sus claves, o cuando se requieren las operaciones de NavigableMap (por ejemplo, subMap, ceilingKey).²⁶

4.4 Práctica de Eficiencia: Iteración Óptima de Map

Existen múltiples formas de iterar sobre un Map, pero una es significativamente más eficiente que las demás.⁶

Enfoque Ineficiente (usando keySet()):

Este enfoque implica iterar sobre el conjunto de claves y, dentro del bucle, usar get() para recuperar cada valor.³³

Java

```
// INEFICIENTE
Map<String, Integer> map = new HashMap<>();
//... poblar el mapa
for (String key : map.keySet()) {
    Integer value = map.get(key); // ¡Segunda búsqueda O(1) o O(log n)!
    //... hacer algo con key y value
}
```

Este método es lento porque realiza dos operaciones de búsqueda por cada entrada: la primera para obtener la clave del iterador keySet() y una segunda búsqueda completa (get()) en el Map para recuperar el valor.³³

Enfoque Eficiente (usando entrySet()):

El enfoque óptimo es iterar sobre el entrySet() del mapa.³³

Java

```
// EFICIENTE
Map<String, Integer> map = new HashMap<>();
//... poblar el mapa
```

```
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    String key = entry.getKey();
    Integer value = entry.getValue();
    //... hacer algo con key y value
}
```

entrySet() devuelve un Set de objetos Map.Entry.⁷ Cada objeto Entry ya *contiene* tanto la clave como el valor.³³ La iteración solo atraviesa la estructura de datos *una vez*, y acceder a entry.getKey() y entry.getValue() es una simple recuperación de campo $O(1)$.³³

5.0 Análisis de Queue y Deque: Estructuras para Procesamiento

Estas colecciones están diseñadas para el procesamiento de elementos, donde los elementos se agregan en un extremo y se eliminan del otro.

5.1 java.util.Queue y java.util.PriorityQueue

La interfaz Queue define un contrato FIFO (First-In, First-Out).¹¹ Sus métodos vienen en dos formas: una que lanza una excepción en caso de fallo (por ejemplo, add(), remove()) y otra que devuelve un valor especial (offer() devuelve false, poll() devuelve null).³⁵

Una implementación clave es **PriorityQueue**, que es una excepción importante a la regla FIFO.³⁵ Es un *heap* (montículo).

- **Diferencia Clave:** Los elementos no se ordenan por inserción, sino por "prioridad" (orden natural o un Comparator).³⁵
- **Comportamiento:** La operación poll() *siempre* devuelve el elemento con la prioridad más alta (por ejemplo, el número más pequeño o el evento más urgente), independientemente de cuándo se agregó.³⁶
- **Caso de Uso:** Sistemas de programación de tareas, procesamiento de eventos por prioridad y algoritmos de grafos (como Dijkstra).

5.2 java.util.Deque: La Solución Unificada para Pilas y Colas

Deque significa "cola de doble extremo" y permite la inserción/eliminación en *ambos* extremos (cabeza y cola).¹¹ Mientras que LinkedList implementa Deque¹¹, una implementación de array, ArrayDeque, es generalmente preferida.

5.3 Práctica de Eficiencia: ArrayDeque vs. Stack y LinkedList

Históricamente, Java usaba dos clases separadas: Stack (para LIFO - Last-In, First-Out) y LinkedList (para FIFO). Ambas han sido reemplazadas en la práctica moderna por ArrayDeque.

- **El Problema de Stack:** La clase `java.util.Stack` (heredada de Java 1.0) extiende `Vector`. Esto significa que cada uno de sus métodos (`push()`, `pop()`) está `synchronized`. Esta sincronización impone una sobrecarga de rendimiento significativa e innecesaria en aplicaciones de un solo hilo.
- **La Solución ArrayDeque:** `ArrayDeque` es una implementación de `Deque` que utiliza un array redimensionable.³⁸ Es la implementación preferida tanto para pilas como para colas.³⁸
 - **Como Pila (LIFO):** Use `push(E e)` (agrega al frente) y `pop()` (elimina del frente).³⁸
 - **Como Cola (FIFO):** Use `offer(E e)` (agrega al final) y `poll()` (elimina del frente).³⁸

`ArrayDeque` es **más rápido que Stack** cuando se usa como pila (porque no tiene sobrecarga de sincronización) y **más rápido que LinkedList** cuando se usa como cola.³⁸ `ArrayDeque` se beneficia de la localidad de caché de su array subyacente, mientras que `LinkedList` debe crear un nuevo objeto `Node` y gestionar punteros para cada inserción, lo que genera más recolección de basura y peor localidad de caché.

6.0 Escribir Aplicaciones Eficientes (Parte 1): Optimización de Parámetros

Una de las optimizaciones de rendimiento más impactantes y a menudo ignorada es la configuración de los parámetros de construcción de una colección.

6.1 El Costo Oculto del Redimensionamiento: `ArrayList` `initialCapacity`

- **El Problema:** Por defecto, un `ArrayList` se crea con una capacidad interna pequeña. Cuando esta capacidad se excede, el `ArrayList` debe realizar una operación de "redimensionamiento": asignar un nuevo array (más grande) y *copiar* todos los elementos existentes del array antiguo al nuevo.⁸ Esta operación de copia es costosa, consume CPU y tiempo.³⁹
- **Análisis de Rendimiento:**
 - *Sub-especificación (`initial < final`):* Este es el peor escenario de rendimiento. Si se crea un `ArrayList` con la capacidad por defecto de 10 y se le añaden 1,000,000 de elementos, se provocarán múltiples operaciones de copia de array, cada una más grande que la anterior.⁴⁰

- *Sobre-especificación (initial > final)*: Un escenario mucho mejor. Hay un pequeño costo inicial por asignar más memoria, pero *no* hay costos de redimensionamiento ni de copia.⁴⁰
- **Mejor Práctica Crítica**: Si se conoce (o se puede estimar razonablemente) el tamaño final de la List de antemano, **siempre** se debe especificar la initialCapacity en el constructor.¹⁴

Java

// Malo: provocará múltiples redimensionamientos costosos

```
List<String> list = new ArrayList<>();
for (int i = 0; i < 10000; i++) {
    list.add(String.valueOf(i));
}
```

// Bueno: Cero redimensionamientos

```
List<String> list = new ArrayList<>(10000);
for (int i = 0; i < 10000; i++) {
    list.add(String.valueOf(i));
}
```

6.2 HashMap: initialCapacity y loadFactor

HashMap tiene un costo de redimensionamiento aún mayor, conocido como *rehashing*.

- **Terminología**⁴¹:
 - Capacity (Capacidad): El número de *buckets* (cubetas) en la tabla hash. Por defecto es 16.
 - Load Factor (Factor de Carga): Un porcentaje (por defecto 0.75f) que determina qué tan lleno puede estar el mapa antes de redimensionarse.
 - Threshold (Umbral): El punto de redimensionamiento. Se calcula como $\text{Threshold} = \text{Capacity} * \text{Load Factor}$. (Por defecto: $16 * 0.75 = 12$).
- **El Proceso de Rehashing**: Cuando el número de entradas (size()) supera el Threshold, el HashMap debe redimensionarse. Típicamente, duplica su capacidad. Luego, debe recalcular el bucket para *cada entrada existente* y redistribuirlas en la nueva tabla más grande. Este es un proceso $O(n)$ muy costoso.⁴²
- **Mejor Práctica Crítica**: Al igual que con ArrayList, si se conoce el número de elementos (n), se debe establecer la capacidad inicial para evitar el *rehashing*. Para prevenir *cualquier* rehashing, la initialCapacity debe establecerse en $(n / \text{loadFactor}) + 1$.
 - *Ejemplo*: Para almacenar 1,000,000 de entradas de forma segura sin rehashing (usando el loadFactor por defecto de 0.75):
 - Capacidad necesaria = $\lceil (1000000 / 0.75f) + 1 \rceil = 1,333,334$
 - `Map<String, Data> map = new HashMap<>(1333334);`

- Esto asegura que el umbral inicial esté por encima de 1,000,000, y el costoso proceso de rehashing nunca se ejecutará.

7.0 Escribir Aplicaciones Eficientes (Parte 2): Concurrencia y Sincronización

Las colecciones estándar en `java.util` (`ArrayList`, `HashMap`, etc.) no son seguras para hilos (*thread-safe*).³⁸ Su uso en un entorno multihilo sin la sincronización adecuada es una receta para el desastre.

7.1 El Peligro: `ConcurrentModificationException`

Si un hilo modifica una colección (p.ej., `list.add()`) mientras otro hilo está iterando sobre ella, el iterador detectará esta modificación y lanzará una `ConcurrentModificationException`.⁴⁴ Este es un comportamiento *fail-fast* (falla rápida), diseñado para alertar sobre un estado concurrente inseguro.³⁸

7.2 El Enfoque Antiguo: `Collections.synchronizedMap` (El Cuello de Botella)

El enfoque tradicional para hacer un `Map` seguro para hilos era envolverlo con un decorador:
`Map<K, V> safeMap = Collections.synchronizedMap(new HashMap<K, V>());`
Este enfoque es problemático y debe evitarse en aplicaciones de alto rendimiento. Es un *wrapper* (envoltorio) que aplica un único bloqueo (un *monitor*) a toda la colección.⁴⁵

- **Problema 1: Baja Escalabilidad:** Solo un hilo puede acceder al mapa a la vez, para cualquier operación (lectura o escritura). Si el Hilo A está leyendo (`get()`), los Hilos B, C y D deben esperar. Esto serializa el acceso y crea un cuello de botella masivo.⁴⁶
- **Problema 2: Iteración Insegura:** A pesar de ser "sincronizado", la iteración (p.ej., `for (K key : safeMap.keySet())`) aún requiere sincronización manual externa (`synchronized(safeMap) {... }`) para prevenir una `ConcurrentModificationException`.⁴⁷

7.3 El Estándar Moderno: `java.util.concurrent.ConcurrentHashMap`

`ConcurrentHashMap` (parte del paquete `java.util.concurrent`) es la implementación preferida para `Map` en entornos concurrentes.⁴⁸

- **Mecanismo de Bloqueo:** No utiliza un bloqueo global. Utiliza un sofisticado sistema de

bloqueo granular (fino).⁴⁶ Las operaciones de escritura bloquean solo el *bucket* o segmento que están modificando, no todo el mapa.⁴⁵ Las operaciones de lectura (*get()*) son generalmente *libres de bloqueo* (no bloqueantes).⁴⁵

- **Ventaja 1: Alta Escalabilidad:** Múltiples hilos pueden leer y escribir en el mapa concurrentemente, siempre que no estén en el mismo bucket. El rendimiento supera drásticamente al de *synchronizedMap* en escenarios del mundo real.⁴⁵
- **Ventaja 2: Iterador *Fail-Safe*:** El iterador devuelto por *ConcurrentHashMap* es *débilmente consistente* (weakly consistent).⁴⁷ **Nunca lanza *ConcurrentModificationException*.**⁴⁵
- **Restricción:** No permite claves ni valores null.⁴⁵

Tabla 7.1: *synchronizedMap* vs. *ConcurrentHashMap*

Característica	<i>Collections.synchronizedMap</i>	<i>ConcurrentHashMap</i>
Nivel de Bloqueo	Global (toda la colección) ⁴⁶	Granular (por bucket/segmento) ⁴⁶
Rendimiento	Pobre (cuello de botella) ⁴⁶	Excelente (alta concurrencia) ⁴⁵
Iterador	<i>Fail-Fast</i> (Lanza CME) ⁴⁵	<i>Fail-Safe</i> (Nunca lanza CME) ⁴⁶
Sinc. de Iteración	Requiere bloqueo manual externo ⁴⁷	No requiere bloqueo externo
Permite Nulos	Depende del mapa subyacente ⁴⁵	No (ni claves ni valores) ⁴⁵

8.0 Escribir Aplicaciones Eficientes (Parte 3): Principios de Diseño de Software

Elegir la colección correcta es solo la mitad de la batalla. Escribir código eficiente y mantenible también requiere adherirse a principios de diseño de software probados.

8.1 Flexibilidad: Programar contra Interfaces

Un principio fundamental de la programación orientada a objetos es "programar contra una interfaz, no contra una implementación".⁴⁹

- **Mal (Código Frágil):** Declarar variables usando la clase concreta.

```
private ArrayList<User> userList = new ArrayList<>();
public ArrayList<User> getUsers() { ... }
```

- Bien (Código Flexible): Declarar variables y tipos de retorno usando la interfaz.⁴

```
private List<User> userList = new ArrayList<>();
public List<User> getUsers() {... }
```

Al depender solo del contrato de la interfaz List⁵⁰, el código *llamador* no se preocupa por los detalles de la implementación.⁴⁹ El propietario de la clase userList es libre de cambiar la implementación interna (por ejemplo, de ArrayList a CopyOnWriteArrayList por razones de concurrencia) en el futuro, sin romper el contrato con ninguno de sus llamadores. Esto promueve el bajo acoplamiento y la mantenibilidad a largo plazo.⁴

8.2 Seguridad: El Rol de los Genéricos (Generics)

Antes de Java 5, las colecciones almacenaban tipos Object (conocidos como tipos crudos o *raw types*).⁵¹ Esto era peligroso.

- **Código Antiguo (Peligroso):**

```
Java
List names = new ArrayList();
names.add("Alice");
names.add(123); // Error lógico, pero permitido
//... más tarde en el código...
String s = (String) names.get(1); // Falla en RUNTIME: ClassCastException
```

- **Código Moderno (Seguro):** Los genéricos (<E>, <K, V>) proporcionan *seguridad de tipos en tiempo de compilación*.⁵²

```
Java
List<String> names = new ArrayList<>();
names.add("Alice");
// names.add(123); // ¡ERROR DE COMPILACIÓN!
String s = names.get(0); // No se necesita casting [52]
```

El uso de genéricos es fundamental para la robustez del software. Mueve la detección de errores de tipo del *runtime* (donde falla en producción frente a un cliente) al *tiempo de compilación* (donde el desarrollador lo corrige).⁵³ El uso de tipos crudos (List en lugar de List<String>) se considera una mala práctica peligrosa en el código moderno.⁵¹

9.0 Conclusiones y Guía de Decisión Rápida

La elección de la colección java.util correcta es fundamental para el rendimiento y la mantenibilidad de una aplicación Java. La decisión debe basarse en los requisitos funcionales (orden, unicidad) y los perfiles de rendimiento (acceso, inserción, concurrencia).

La siguiente es una guía de decisión arquitectónica basada en este análisis:

1. **¿Necesita una List (ordenada, permite duplicados)?**
 - **Por defecto:** Use **ArrayList**.¹⁶
 - *Optimización:* Si conoce el tamaño, use `new ArrayList<>(size)`.³⁹
2. **¿Necesita un Set (único, sin duplicados)?**
 - **Por defecto (sin orden):** Use **HashSet**.²³
 - **Si necesita orden de inserción:** Use **LinkedHashSet**.²³
 - **Si necesita ordenación natural (A-Z):** Use **TreeSet**.²³
3. **¿Necesita un Map (mapeo clave-valor)?**
 - **Por defecto (sin orden):** Use **HashMap**.³⁰
 - *Optimización:* Si conoce el tamaño (n), use `new HashMap<>((int)(n / 0.75f) + 1)`.⁴²
 - **Si necesita orden de inserción:** Use **LinkedHashMap**.³⁰
 - **Si necesita ordenación por clave:** Use **TreeMap**.³⁰
4. **¿Necesita una Pila (LIFO) o una Cola (FIFO)?**
 - **Respuesta Unificada:** Use **ArrayDeque**.³⁸
 - *Evite:* Stack (obsoleto, lento) y LinkedList (más lento que ArrayDeque).
5. **¿Necesita una Cola de Prioridad?**
 - Use **PriorityQueue**.³⁶
6. **¿Necesita una colección Segura para Hilos (Thread-Safe)?**
 - **Respuesta Única:** Use **ConcurrentHashMap**.⁴⁵
 - *Evite:* Collections.synchronizedMap (lento, es un cuello de botella).⁴⁶
7. **¿Cómo debe declarar sus colecciones?**
 - **Principio 1:** Programe contra la *Interfaz*: `List<String> list = ...`.⁴
 - **Principio 2:** Use *Genéricos* para seguridad de tipos: `... = new ArrayList<String>()`.⁵³

Fuentes citadas

1. Java Collections Tutorial - GeeksforGeeks, acceso: noviembre 9, 2025, <https://www.geeksforgeeks.org/java/java-collection-tutorial/>
2. Collections in Java - Everything You MUST Know | DigitalOcean, acceso: noviembre 9, 2025, <https://www.digitalocean.com/community/tutorials/collections-in-java-tutorial>
3. Outline of the Collections Framework, acceso: noviembre 9, 2025, <https://docs.oracle.com/javase/8/docs/technotes/guides/collections/reference.html>
4. Interfaz List en Java. La interfaz Collection es la raíz de la... | by ..., acceso: noviembre 9, 2025, <https://medium.com/@barbieri.santiago/interfaz-list-en-java-fbb63beea8e7>
5. Difference between HashMap and ArrayList in Java? - Stack Overflow, acceso: noviembre 9, 2025, <https://stackoverflow.com/questions/2395814/difference-between-hashmap-and>

[-arraylist-in-java](#)

6. Working With Maps Using Streams | Baeldung, acceso: noviembre 9, 2025, <https://www.baeldung.com/java-maps-streams>
7. Iterate Over a Map in Java | Baeldung, acceso: noviembre 9, 2025, <https://www.baeldung.com/java-iterate-map>
8. Which Java Collection should I use? - Stack Overflow, acceso: noviembre 9, 2025, <https://stackoverflow.com/questions/21974361/which-java-collection-should-i-use>
9. ▷ Set en Java → 【 Tutorial de JAVA 】 - Oregoom.com, acceso: noviembre 9, 2025, <https://oregoom.com/java/set/>
10. Día-89 Entendiendo Set en Java - Q2B Studio, acceso: noviembre 9, 2025, <https://www.q2bstudio.com/nuestro-blog/18788/dia-89-entendiendo-set-en-java>
11. Java Queue Interface y sus implementaciones - CodeGym, acceso: noviembre 9, 2025, <https://codegym.cc/es/groups/posts/es.307.java-queue-interface-y-sus-implementaciones>
12. Time Complexity of Java Collections API | by Bikash Dubey - Medium, acceso: noviembre 9, 2025, <https://bikashdubey42.medium.com/arraylist-vs-linkedlist-in-java-633e68f49d22>
13. ArrayList vs LinkedList in Java - GeeksforGeeks, acceso: noviembre 9, 2025, <https://www.geeksforgeeks.org/java/arraylist-vs-linkedlist-java/>
14. ArrayList (Java Platform SE 8) - Oracle Help Center, acceso: noviembre 9, 2025, <https://docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html>
15. ArrayList & LinkedList Big-O Complexities - Stack Overflow, acceso: noviembre 9, 2025, <https://stackoverflow.com/questions/74766925/arraylist-linkedlist-big-o-complexities>
16. When to use LinkedList over ArrayList in Java? - Stack Overflow, acceso: noviembre 9, 2025, <https://stackoverflow.com/questions/322715/when-to-use-linkedlist-over-arraylist-in-java>
17. Choosing the Right Java Collection | Baeldung, acceso: noviembre 9, 2025, <https://www.baeldung.com/java-choose-list-set-queue-map>
18. When to use ArrayList V/S LinkedList? In terms of Big O Notation - YouTube, acceso: noviembre 9, 2025, <https://www.youtube.com/watch?v=MrlpQZ4tpT4>
19. LinkedList vs ArrayList in Java Tutorial - Which Should You Use? - YouTube, acceso: noviembre 9, 2025, <https://www.youtube.com/watch?v=5dscMs2hnDI>
20. Performance of contains() in a HashSet vs ArrayList | Baeldung, acceso: noviembre 9, 2025, <https://www.baeldung.com/java-hashset-arraylist-contains-performance>
21. HashSet vs LinkedHashSet vs TreeSet in Java | by Ramesh Fadatare - Medium, acceso: noviembre 9, 2025, <https://rameshfadatare.medium.com/hashset-vs-linkedhashset-vs-treeset-in-java-ceb161dbf258>
22. Sets, HashSet, TreeSet, LinkedHashSet - Full Tutorial - Java Data Structure

- Tutorial, acceso: noviembre 9, 2025,
<https://www.youtube.com/watch?v=JpOScjrGBRg>
23. Difference and similarities between HashSet, LinkedHashSet and TreeSet in Java - GeeksforGeeks, acceso: noviembre 9, 2025,
<https://www.geeksforgeeks.org/java/difference-and-similarities-between-hashset-linkedhashset-and-treeset-in-java/>
 24. Cómo ordenar un set en Java - LabEx, acceso: noviembre 9, 2025,
<https://labex.io/es/tutorials/java-how-to-sort-a-set-in-java-414138>
 25. java - In which cases should I use a HashSet over a TreeSet ..., acceso: noviembre 9, 2025,
<https://stackoverflow.com/questions/1463284/in-which-cases-should-i-use-a-hashset-over-a-treeset>
 26. java - Difference between HashMap, LinkedHashMap and TreeMap ..., acceso: noviembre 9, 2025,
<https://stackoverflow.com/questions/2889777/difference-between-hashmap-linkedhashmap-and-treemap>
 27. Java TreeMap vs HashMap | Baeldung, acceso: noviembre 9, 2025,
<https://www.baeldung.com/java-treemap-vs-hashmap>
 28. ¿Qué es java.util.Map y para qué sirve en Java? - KeepCoding, acceso: noviembre 9, 2025,
<https://keepcoding.io/blog/que-es-java-util-map-y-para-que-sirve-en-java/>
 29. HashMap vs TreeMap in Java | Stack a Byte, acceso: noviembre 9, 2025,
<https://stackabyte.com/tutorials/java/java-hashmap-treemap-comparison>
 30. Differences between TreeMap, HashMap and LinkedHashMap in Java - GeeksforGeeks, acceso: noviembre 9, 2025,
<https://www.geeksforgeeks.org/java/differences-treemap-hashmap-linkedhashmap-java/>
 31. Interfaz de mapa en Java - CodeGym, acceso: noviembre 9, 2025,
<https://codegym.cc/es/groups/posts/es.221.interfaz-de-mapa-en-java>
 32. Guía completa de Map en Java: de principiante a avanzado - Q2B Studio, acceso: noviembre 9, 2025,
<https://www.q2bstudio.com/nuestro-blog/24613/guia-completa-de-map-en-java-de-principiante-a-avanzado>
 33. Using the Map.Entry Java Class | Baeldung, acceso: noviembre 9, 2025,
<https://www.baeldung.com/java-map-entry>
 34. Three data structures: queue, stack, and deque, acceso: noviembre 9, 2025,
<https://www.cs.cornell.edu/courses/JavaAndDS/files/queueStack.pdf>
 35. ▷ Queue en Java → 【 Tutorial de JAVA 】 - Oregoom.com, acceso: noviembre 9, 2025,
<https://oregoom.com/java/queue/>
 36. Queue, Deque, and Priority Queue: Key Data Structures Explained - Omega.py, acceso: noviembre 9, 2025,
<https://www.alexomegapy.com/post/queue-deque-and-priority-queue-key-data-structures-explained>
 37. PriorityQueue (Java Platform SE 8) - Oracle Help Center, acceso: noviembre 9, 2025,
<https://docs.oracle.com/javase/8/docs/api/java/util/PriorityQueue.html>

38. ArrayDeque (Java Platform SE 8) - Oracle Help Center, acceso: noviembre 9, 2025, <https://docs.oracle.com/javase/8/docs/api/java/util/ArrayDeque.html>
39. The Capacity of an ArrayList vs the Size of an Array in Java | Baeldung, acceso: noviembre 9, 2025, <https://www.baeldung.com/java-list-capacity-array-size>
40. Performance Evaluation of JAVA ArrayList | by Malith Jayasinghe ..., acceso: noviembre 9, 2025, <https://medium.com/@malith.jayasinghe/a-performance-evaluation-of-java-arraylist-f097582b5c4d>
41. HashMap Under the Hood | Baeldung, acceso: noviembre 9, 2025, <https://www.baeldung.com/java-hashmap-advanced>
42. Java HashMap Load Factor | Baeldung, acceso: noviembre 9, 2025, <https://www.baeldung.com/java-hashmap-load-factor>
43. Diving Into the Internals of Java HashMap — Part I — Basics Concepts and Core Methods, acceso: noviembre 9, 2025, <https://medium.com/@sorbkhandelwal/diving-into-the-internals-of-java-hashmap-part-i-basics-concepts-and-core-methods-d2070bb0768c>
44. LinkedList (Java Platform SE 8) - Oracle Help Center, acceso: noviembre 9, 2025, <https://docs.oracle.com/javase/8/docs/api/java/util/LinkedList.html>
45. Collections.synchronizedMap vs. ConcurrentHashMap | Baeldung, acceso: noviembre 9, 2025, <https://www.baeldung.com/java-synchronizedmap-vs-concurrenthashmap>
46. java - ConcurrentHashMap vs Synchronized HashMap - Stack Overflow, acceso: noviembre 9, 2025, <https://stackoverflow.com/questions/1291836/concurrenthashmap-vs-synchronized-hashmap>
47. What's the difference between ConcurrentHashMap and Collections.synchronizedMap(Map)? - Codemia, acceso: noviembre 9, 2025, https://codemia.io/knowledge-hub/path/whats_the_difference_between_concurrenthashmap_and_collectionsynchronizedmapmap
48. Unlocking Concurrent Power: A Guide to java.util.concurrent Pt. 1 - Medium, acceso: noviembre 9, 2025, <https://medium.com/@alxkm/unlocking-concurrent-power-a-guide-to-java-util-concurrent-pt-1-b1342edadad1>
49. ¿Por qué necesitamos una interfaz Java cuando todavía tenemos que implementar los métodos cada vez que los usamos? - Quora, acceso: noviembre 9, 2025, <https://es.quora.com/Por-qu%C3%A9-necesitamos-una-interfaz-Java-cuando-todav%C3%ADa-tenemos-que-implementar-los-m%C3%A9todos-cada-vez-que-los-usamos>
50. Colecciones en Java - CodeGym, acceso: noviembre 9, 2025, <https://codegym.cc/es/groups/posts/es.837.colecciones-en-java>
51. Java - Generic types and collections - Stack Overflow, acceso: noviembre 9, 2025, <https://stackoverflow.com/questions/45185619/java-generic-types-and-collections>

52. Type-Safety and Type-Casting in Java Generics - GeeksforGeeks, acceso: noviembre 9, 2025, <https://www.geeksforgeeks.org/java/type-safety-and-type-casting-in-java-generics/>
53. Java Generics: Type Safety and Performance - Medium, acceso: noviembre 9, 2025, <https://medium.com/@AlexanderObregon/java-generics-type-safety-and-performance-a0864ca72a5a>
54. Ensuring Type Safety With Collections.checkedXXX() in Java | Baeldung, acceso: noviembre 9, 2025, <https://www.baeldung.com/java-collections-type-safety>